

Geographically Weighted Regression

Chris Brunsdon

18 August 2015



This work is licensed under the Creative Commons Attribution-ShareAlike 3.0 Unported License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/> (<http://creativecommons.org/licenses/by-sa/3.0/>).



The thinking behind this tutorial

These practicals are designed to have an explanatory text, together with code examples. Note that all code examples have a light grey background, and are boxed. Output from R is also shown, and text output is also boxed. Graphical output is shown 'in line'. The idea is to copy the text in the boxes into a running version of R - you can use the output to see whether you are on the right track. Also, don't be afraid to experiment - it may not always work, but for example playing around with some of the function parameters change some aspects of the analysis, or of the graphical presentation. Also, it is generally assumed that the code you type in from the boxes is actually entered in the same order as it appears here. Some of the later boxes depend on what was done earlier, and so skipping ahead might lead to errors. The help facility (putting a question mark in front of a command, for example `?plot`, and hitting return) is also a good way to discover things...

To get started with this tutorial, first load the `GWmodel` package.

```
library(GWmodel)
```

A Quick Explanation of Geographically Weighted Regression

Geographically weighted regression (GWR) is a useful tool for exploring spatial heterogeneity in the relationships between variables. A typical ordinary least squares regression calibrates a model of the form

$$y_i = \sum_{ij} \beta_j x_{ij} + \varepsilon_i$$

where

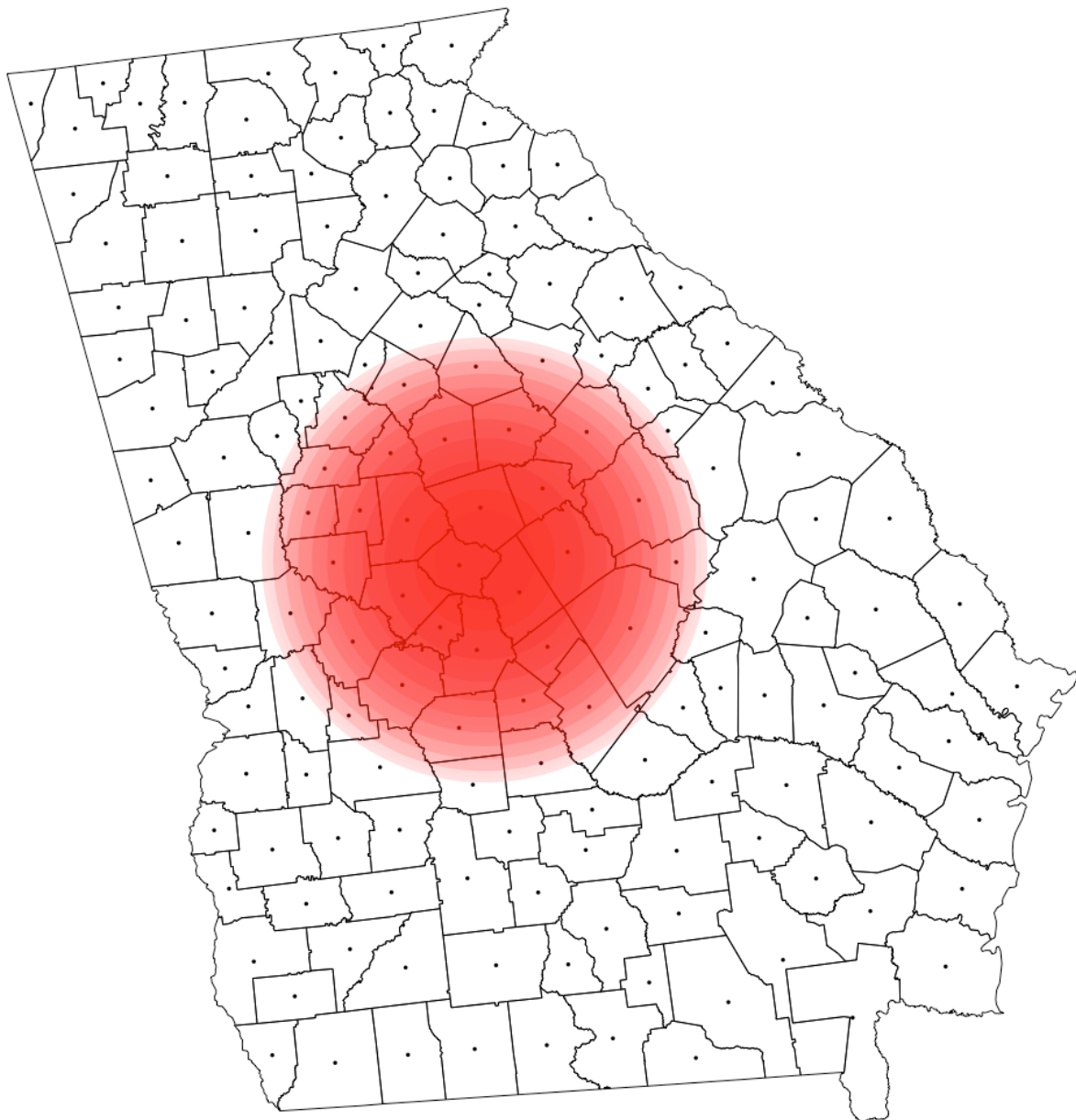
$$\varepsilon_i \sim N(0, \sigma^2)$$

via maximum likelihood. A spatially heterogeneous variant of this model is

$$y_i = \sum_{ij} \beta_j(u_i, v_i) x_{ij} + \varepsilon_i$$

where the constant β_j parameters are replaced by functions $\beta_j(\cdot, \cdot)$ of location (u, v) . An attempt to estimate these could provide some kind of insight into whether spatial heterogeneity in the regression parameters are occurring. In simpler terms we are exploring whether the relationship between the predictor variables and the outcome variable is changing geographically.

One way of doing this is to use a *moving window* approach - this is inspired by the idea of *Loess Smoothing* (Cleveland 1979). To estimate the value of $\beta_j(u, v)$ a regression model is fitted to all of the y_i, x_{ij} data records for observations inside a circle centred on (u, v) having a radius r . Allowing (u, v) to sweep over the study area allows changes in the regression coefficients to be monitored. Finally, to reduce the impact of observations suddenly entering the circle, a weighted regression model is used, with weights gradually reducing from the centre of the circle at (u, v) to a zero value at the perimeter.



The data

In this practical you will be working with house price data, obtained from the Nationwide Building Society (NBS) in England - this is a sample of houses sold in 2001 with mortgages arranged by NBS in and around London. The data is in a `SpatialPolygonsDataFrame` frame called `londonhp`.

```
data(LondonHP)
head(data.frame(londonhp))
```

	PURCHASE	FLOORSZ	TYPEDETCH	TPSEMIDTCH	TYPETRRD	TYPEBNGLW	TYPEFLAT	BLDPWW1	BL	
DPOSTW										
0	157000	77	1	0	0	1	0	0		
0										
1	113500	75	0	0	1	0	0	1		
0										
2	81750	64	0	0	0	0	1	0		
0										
3	150000	95	0	1	0	0	0	0		
0										
4	190000	107	1	0	0	0	0	0		
0										
5	159950	100	0	1	0	0	0	0		
0										
	BLD60S	BLD70S	BLD80S	BLD90S	BATH2	BEDS2	GARAGE1	CENTHEAT	UNEMPLOY	PRO
F BLDINTW										
0	0	0	0	0	0	1	0	1	0.03566768	0.478699
2	1									
1	0	0	0	0	0	1	0	1	0.03566768	0.478699
2	0									
2	0	0	1	0	0	0	1	0	0.03566768	0.478699
2	0									
3	0	0	0	0	0	1	0	1	0.03566768	0.478699
2	1									
4	0	0	0	0	0	1	1	1	0.03566768	0.478699
2	1									
5	0	1	0	0	0	1	1	1	0.02408854	0.477371
5	0									
	coords.x1	coords.x2	optional							
0	533200	159400	TRUE							
1	533300	159700	TRUE							
2	532000	159800	TRUE							
3	531900	160100	TRUE							
4	532800	160300	TRUE							
5	535700	161700	TRUE							

From this, it can be seen that the houses purchase price (`PURCHASE`) and floor area (`FLOORSZ`) are recorded together with a number of other artifacts of the house - for example whether it has a garage (`GARAGE1`). There are also variables relating to social conditions in the neighbourhood of the property - for example the proportion of the working population employed in professional or managerial jobs (`PROF`) .

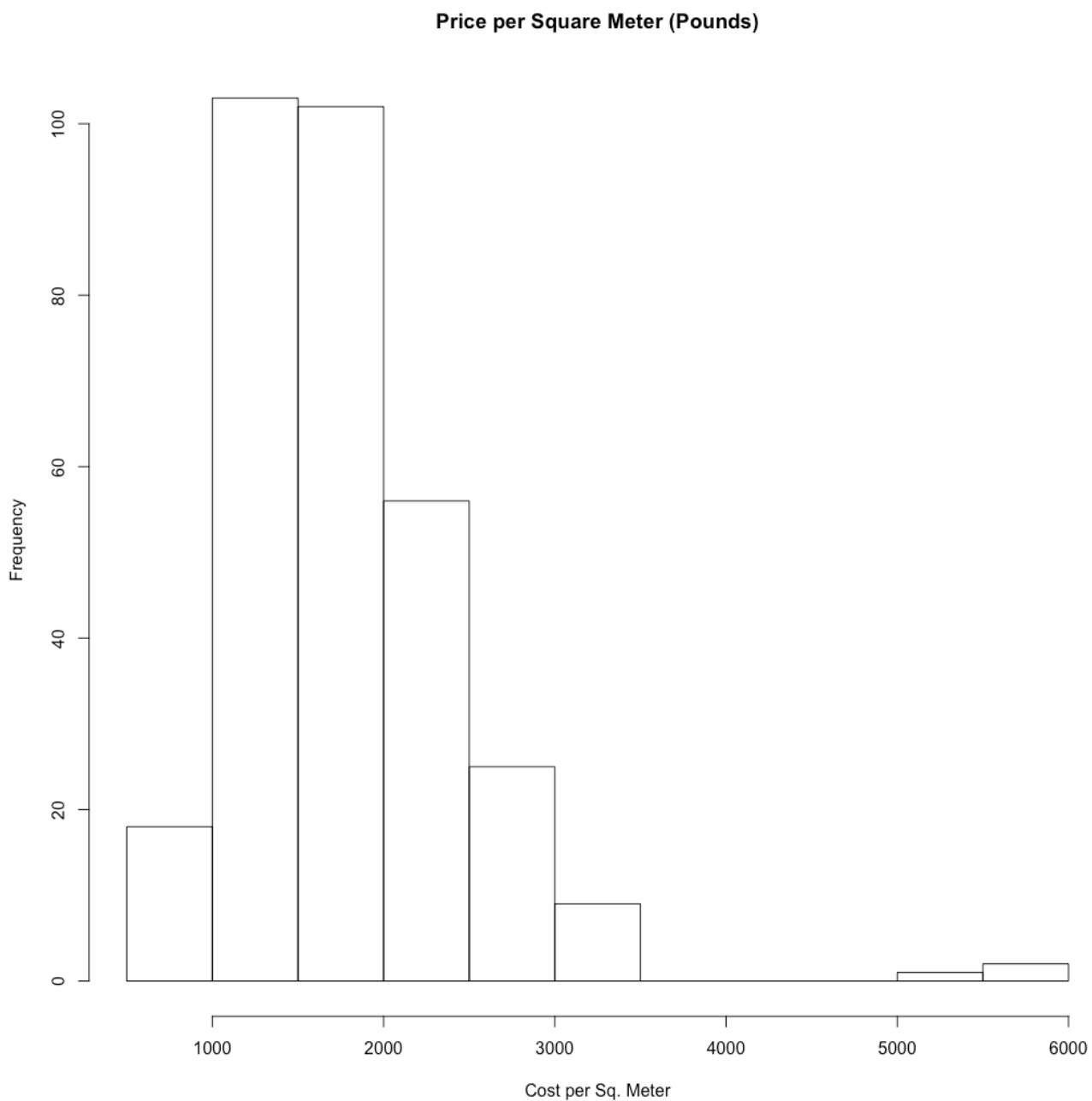
Here, the y -variable will be housing cost per square meter of floor area.

```
londonhp$PPSQM <- londonhp$PURCHASE / londonhp$FLOORSZ
```

Looking at the distribution of the price per square meter can be done using the standard `hist`

command in R:

```
hist(londonhp$PPSQM,main="Price per Square Meter (Pounds)",xlab="Cost per Sq.  
Meter",ylab='Frequency')
```



From this there is a fairly clear distribution shape with a small number of very costly properties (in terms of cost per unit of floor area). We can also look some summary statistics

```
mean(londonhp$PPSQM)
```

```
[1] 1779.542
```

```
sd(londonhp$PPSQM)
```

```
[1] 657.2502
```

That gives the mean and standard deviation - however the skewness doesn't come as standard, but you can get it from the enigmatically-named R library `e1071` :

```
library(e1071)
skewness(londonhp$PPSQM)
```

```
[1] 2.067223
```

From this value of around 2.1 it can be seen that this distribution is strongly positively skewed - suggesting a large upper tail - that is, a central group of typically-priced houses together with a long tail of relatively expensive ones - much as the histogram showed.

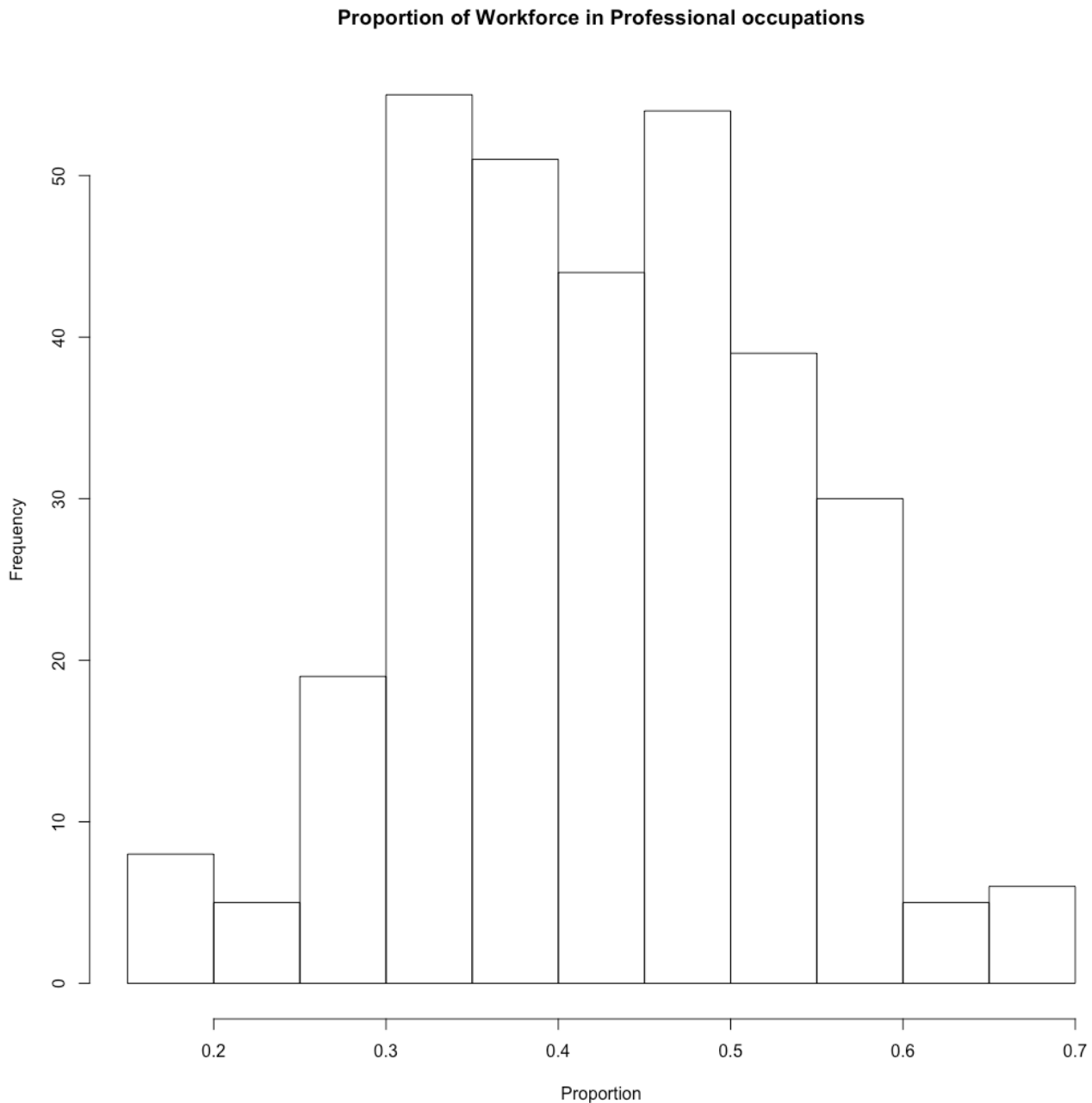
In particular note the skewness again. Skewness is a *dimensionless* quantity, so that it doesn't depend on whether floor areas are in square meters or square feet, or if house prices were converted to Euros or US Dollars. This is unlike means and standard deviations - these take on the units of the data - in this example they are in UK Pounds per square meter. A consequence of this is that it is therefore possible to compare the skewness of two different data sets - they are both on the same (dimensionless) scale. Looking at unemployment, for example:

```
skewness(londonhp$PROF)
```

```
[1] -0.01700381
```

this is very slightly negatively skewed - but perhaps not enough to reject the idea that it has a symmetrical distribution. This is confirmed visually by drawing a histogram.

```
hist(londonhp$PROF,main="Proportion of Workforce in Professional occupation
s",xlab="Proportion",ylab='Frequency')
```



A Relationship in the Data?

One might reasonably expect social conditions in an area to influence property costs. A simple way to investigate this is to regress the professional and managerial employment proportion (scaled so that 1 implies all economically active residents to be employed in professional or managerial jobs) against the price per square meter:

```
linmod <- lm(PPSQM~PROF,data=londonhp) # Store the regression model to use in  
a plot later  
summary(linmod)
```

```
Call:
lm(formula = PPSQM ~ PROF, data = londonhp)

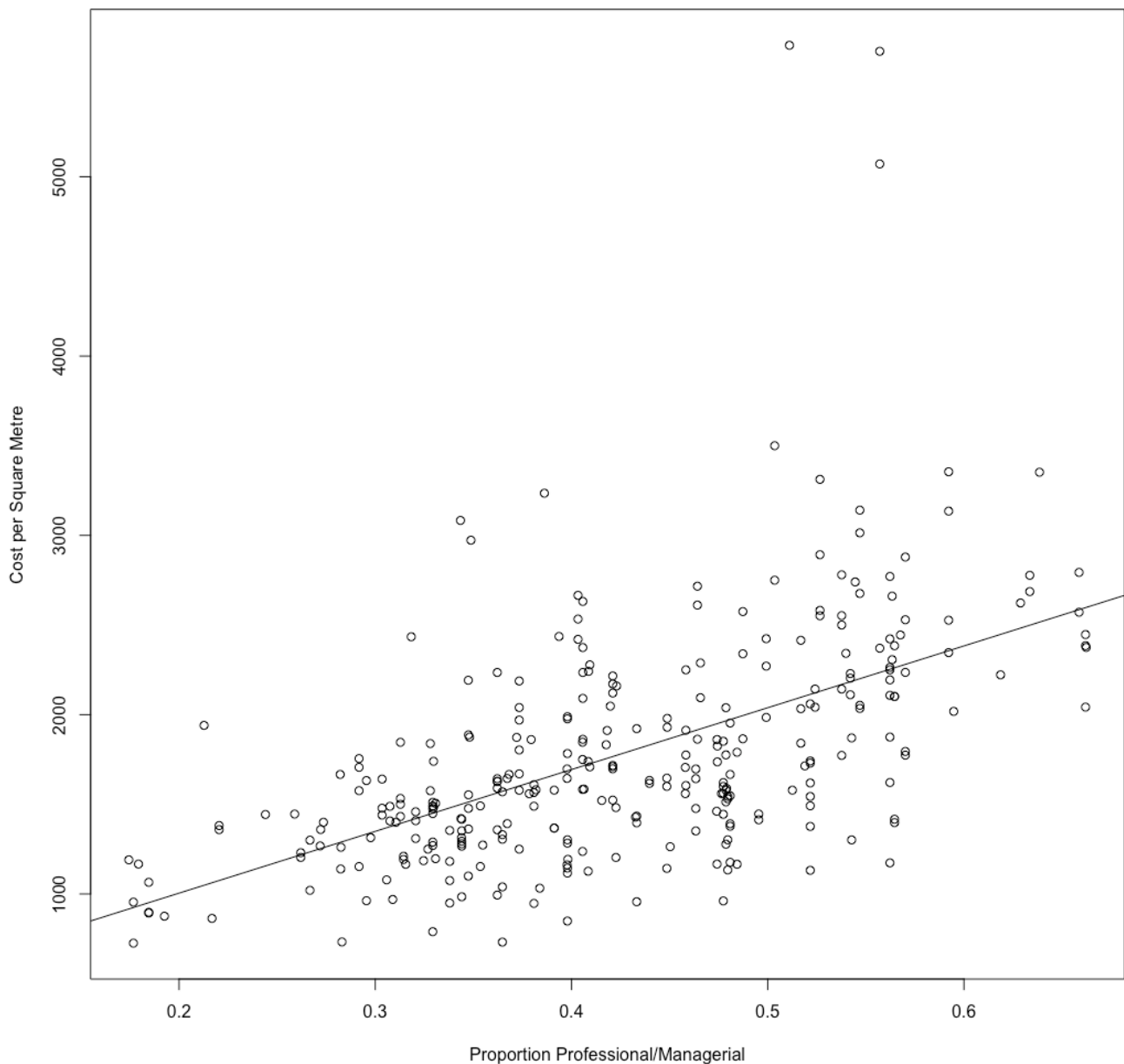
Residuals:
    Min       1Q   Median       3Q      Max
-1079.6  -300.1   -50.6    214.5   3656.7

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)    315.4      128.9    2.447  0.0149 *
PROF          3446.1      294.4   11.704 <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 549.3 on 314 degrees of freedom
Multiple R-squared:  0.3038,    Adjusted R-squared:  0.3015
F-statistic:  137 on 1 and 314 DF,  p-value: < 2.2e-16
```

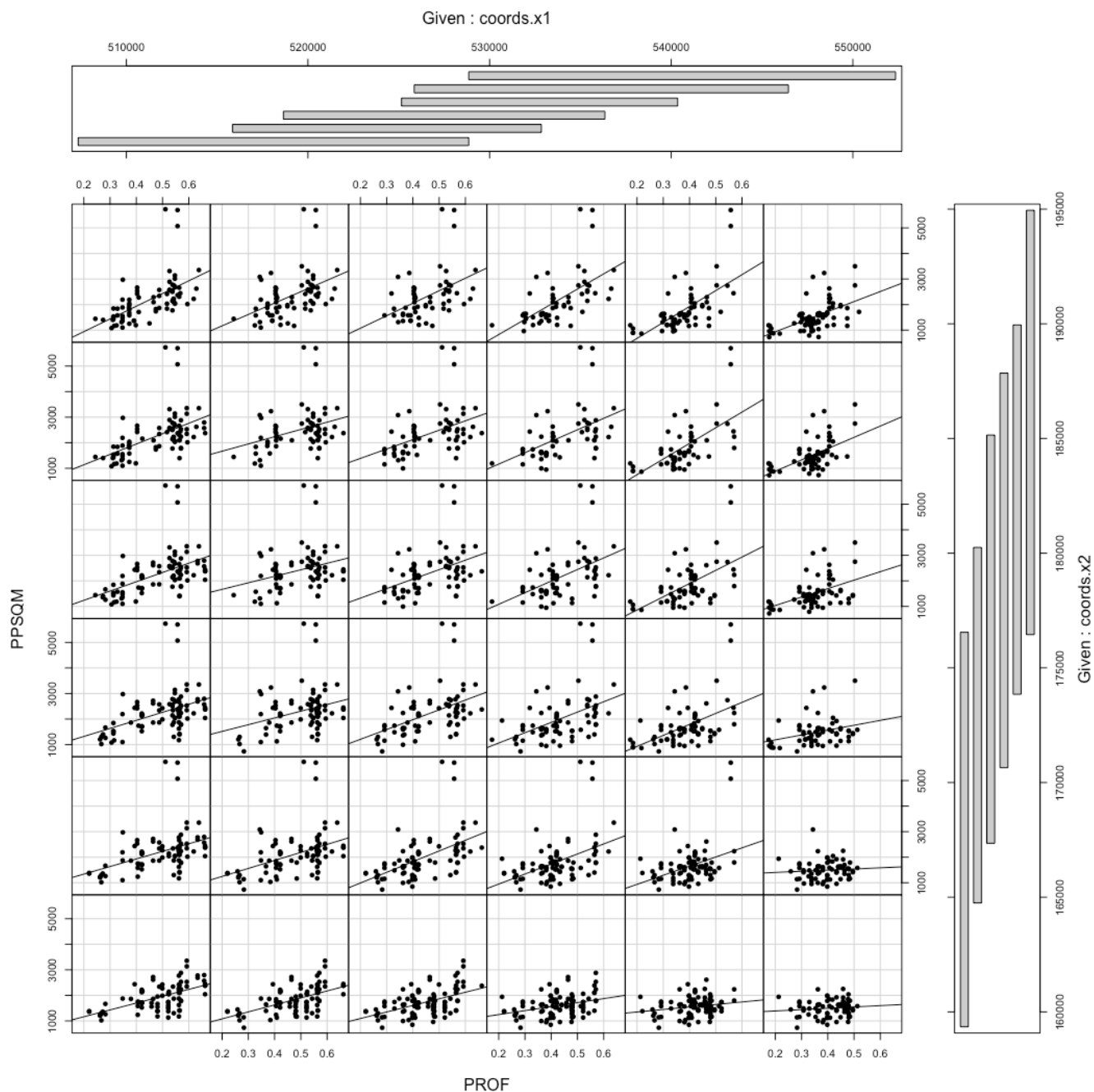
This suggests there is strong evidence of a link between the two variables - perhaps unsurprisingly. This is confirmed visually below:

```
plot(PPSQM~PROF,data=londonhp,xlab='Proportion Professional/Managerial',ylab
='Cost per Square Metre')
abline(linmod)
```



A question that might arise is whether this relationship is the same everywhere in the London area. A quick graphical exploration can be achieved using the *coplot* (Cleveland 1993) -

```
panel.lm <- function(x,y,...) {  
  points(x, y, pch=16)  
  abline(lm(y~x))  
}  
coplot(PPSQM~PROF | coords.x1*coords.x2, data=data.frame(londonhp), panel=panel.lm,  
m, overlap=0.8)
```



very briefly, in the above code the data area divided into subsets on the basis of their locations (stored in `londonhp` as `coords.x1` and `coords.x2` as the eastings and northings). Then:

1. `panel.lm` is a function specifying what is to be done in terms of visualisation for each subset. Here the points are plotted and the regression line is drawn.
2. The `coplot` function actually draws the coplot. `overlap` specifies the proportion of overlap in the subsets - also shown graphically against the rows and columns of the plot matrix.
3. Unfortunately, the data set has to be referenced as `data.frame(londonhp)` - the common trick of treating a `SpatialPointsDataFrame` like a standard data frame doesn't work otherwise.

Looking at the coplot, it seems there is some degree of variability in the relationship between the proportion of professional and managerial workforce and the price per square meter of floor area. However, although the coplot is a useful tool to visualise this, the subsetting process does distort geographical pattern to an extent - and so GWR will also be employed to

look at this.

Geographically Weighted Regression

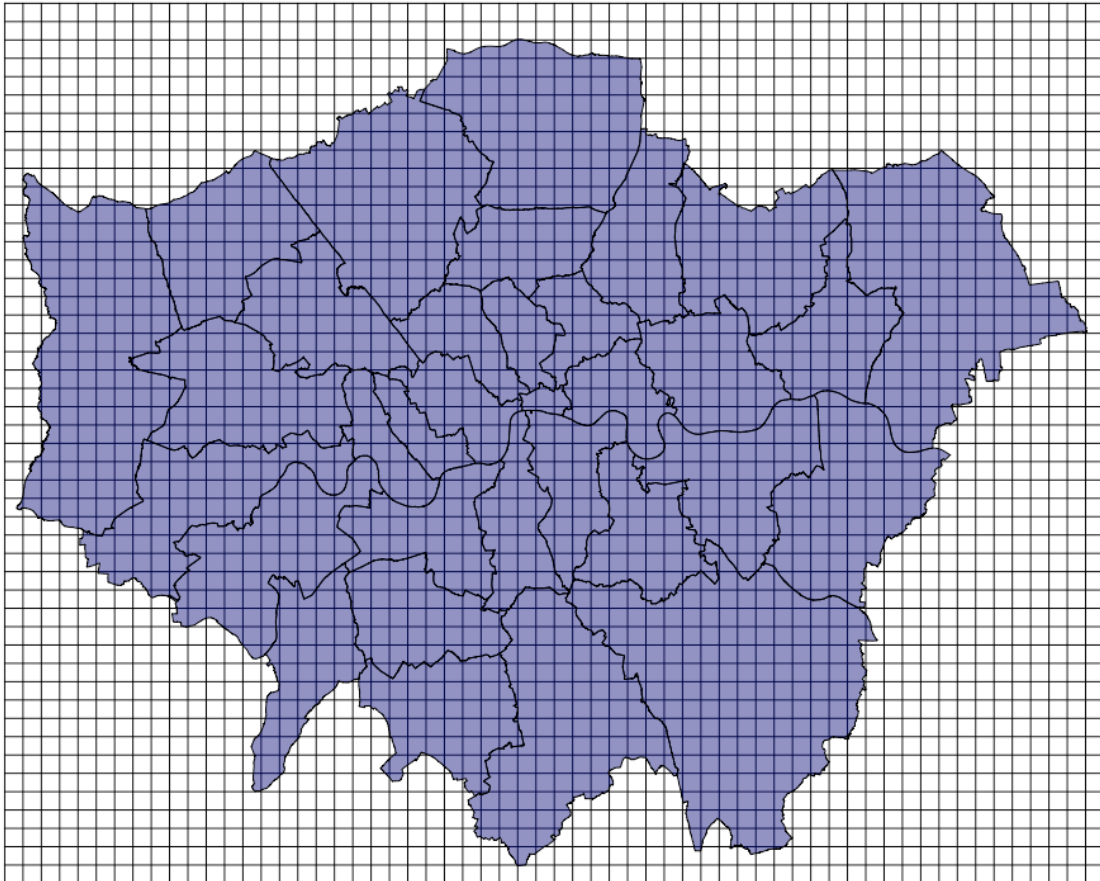
The GWR model is relatively easy to specify - here the `gwr.basic` function is used (from `GWmodel`). The first thing to do is look at the locations of the houses in the data set. It is useful to have the outlines of London boroughs as a backdrop:

```
data(LondonBorough)
plot(londonborough)
plot(londonhp, pch=16, col='firebrick',add=TRUE)
```



The data are rather clustered, but it may still be useful to calibrate the GWR model over a regular grid of observations. The grid is now created:

```
grd <- SpatialGrid(GridTopology(c(503400,155400),c(1000,1000),c(60,48)))  
plot(grd)  
plot(londonborough,add=TRUE,col=adjustcolor('navyblue',alpha.f=0.5))
```



Next, compute the distances between the points on the grid where $\beta_j(u, v)$ will be estimated, and the points where the house prices are observed. This takes a while, but if it is pre-computed, successive applications of GWR functions are speeded up.

```
DM <- gw.dist(dp.locat=coordinates(londonhp),rp.locat=coordinates(grd))
```

Now everything is set up for a basic GWR analysis - this is done using `gwr.basic`

```
gwr.res <- gwr.basic(PPSQM~PROF, data=londonhp, regression.points=grd, bw=1000  
0, dMat=DM,kernel='gaussian')
```

This just creates a `gwr` object, whose print method is nearly the same format as the old output from GWR 3.0 (if you remember that!)

```
gwr.res
```

```
*****
```

```
*                               Package    GWmodel                               *
```

```
*****
```

```
Program starts at: 2016-04-29 14:05:01
```

```
Call:
```

```
gwr.basic(formula = PPSQM ~ PROF, data = londonhp, regression.points = grd,
  bw = 10000, kernel = "gaussian", dMat = DM)
```

```
Dependent (y) variable:  PPSQM
```

```
Independent variables:  PROF
```

```
Number of data points: 316
```

```
*****
```

```
*                               Results of Global Regression                               *
```

```
*****
```

```
Call:
```

```
lm(formula = formula, data = data)
```

```
Residuals:
```

Min	1Q	Median	3Q	Max
-1079.6	-300.1	-50.6	214.5	3656.7

```
Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	315.4	128.9	2.447	0.0149	*
PROF	3446.1	294.4	11.704	<2e-16	***

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 549.3 on 314 degrees of freedom
```

```
Multiple R-squared:  0.3038,    Adjusted R-squared:  0.3015
```

```
F-statistic:   137 on 1 and 314 DF,  p-value: < 2.2e-16
```

```
***Extra Diagnostic information
```

```
Residual sum of squares: 94740638
```

```
Sigma(hat): 549.2921
```

```
AIC: 4887.817
```

```
AICc: 4887.894
```

```
*****
```

```
*                               Results of Geographically Weighted Regression                               *
```

```
*****
```

```
*****Model calibration information*****
```

```
Kernel function: gaussian
```

```
Fixed bandwidth: 10000
```

```
Regression points: A separate set of regression points is used.
```

```
Distance metric: A distance matrix is specified for this model calibration.
```

```
*****Summary of GWR coefficient estimates:*****
      Min. 1st Qu.  Median 3rd Qu.    Max.
Intercept  32.32  310.40  477.10  588.90  976.9
PROF       1342.00 2447.00 3072.00 3681.00 4416.0

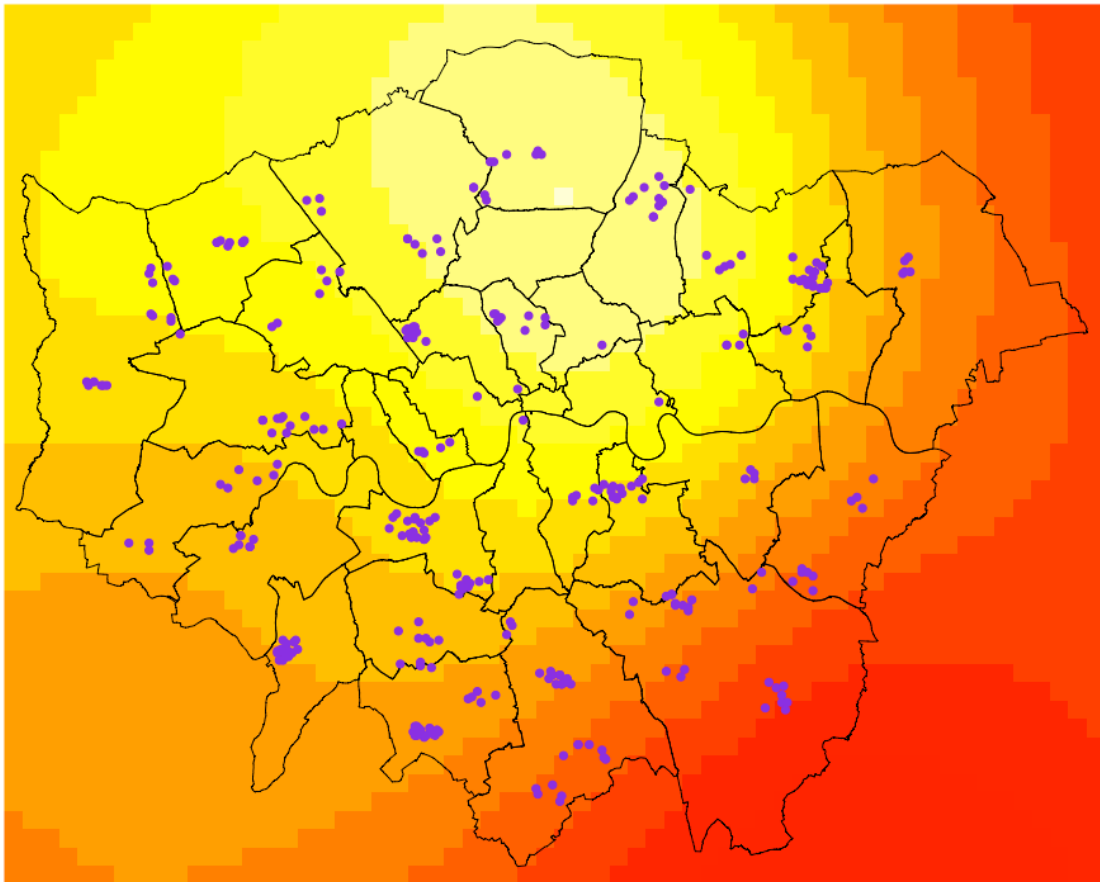
*****
Program stops at: 2016-04-29 14:05:01
```

The *bandwidth* is the width of the circle. Here it is just specified manually as the `bw` argument - here 10km - but it can be chosen 'automatically' by cross-validation. The `kernel` argument specifies the functional form of the kernel (the weighting applied in the window) - here it is Gaussian - so that if h is the bandwidth, then the weight for a point at distance d from (u, v) is

$$e^{-\frac{d^2}{2h^2}}$$

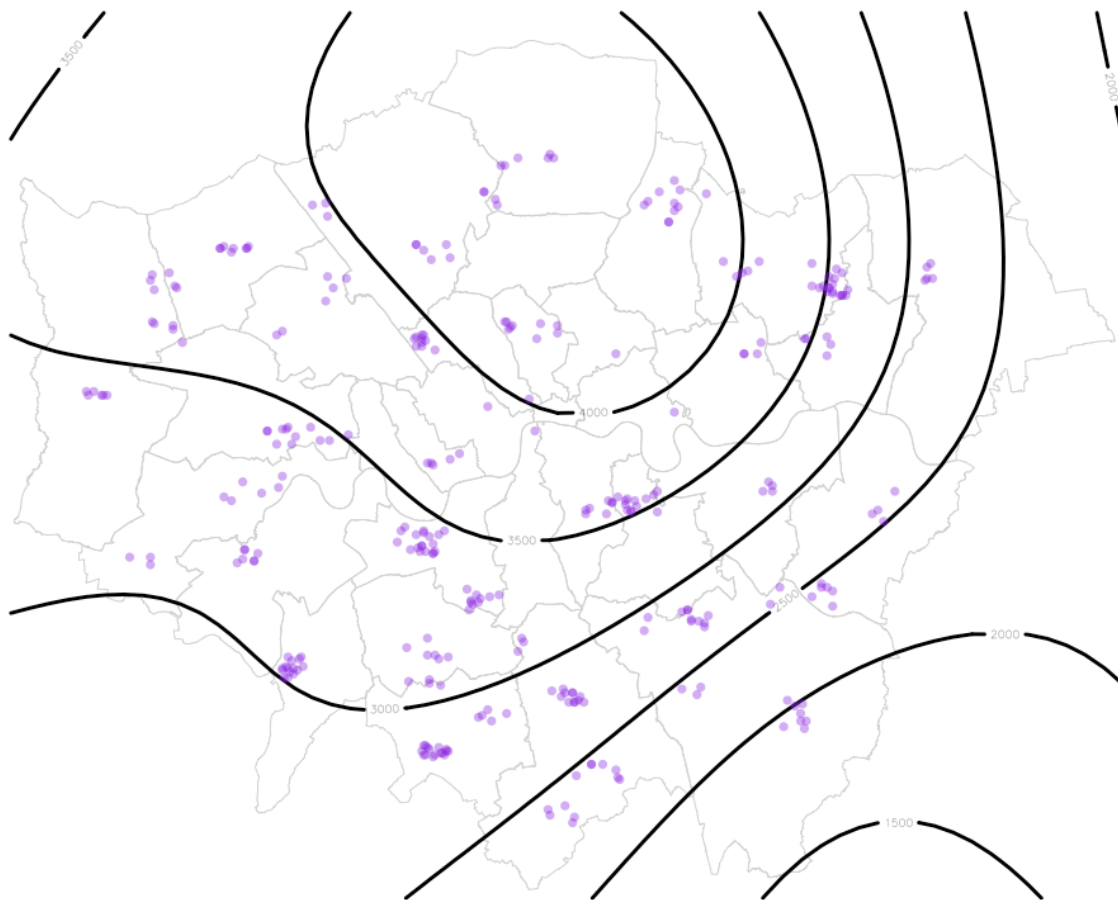
It is also possible to obtain an image from this result (the `Spatial*` object with the GWR results is in `gwr.res$SDF` - here the object is a `SpatialPixelsDataFrame`). This may be used with `image` to view the results here.

```
image(gwr.res$SDF, 'PROF')
plot(londonborough, add=TRUE)
plot(londonhp, add=TRUE, pch=16, col='blueviolet')
```



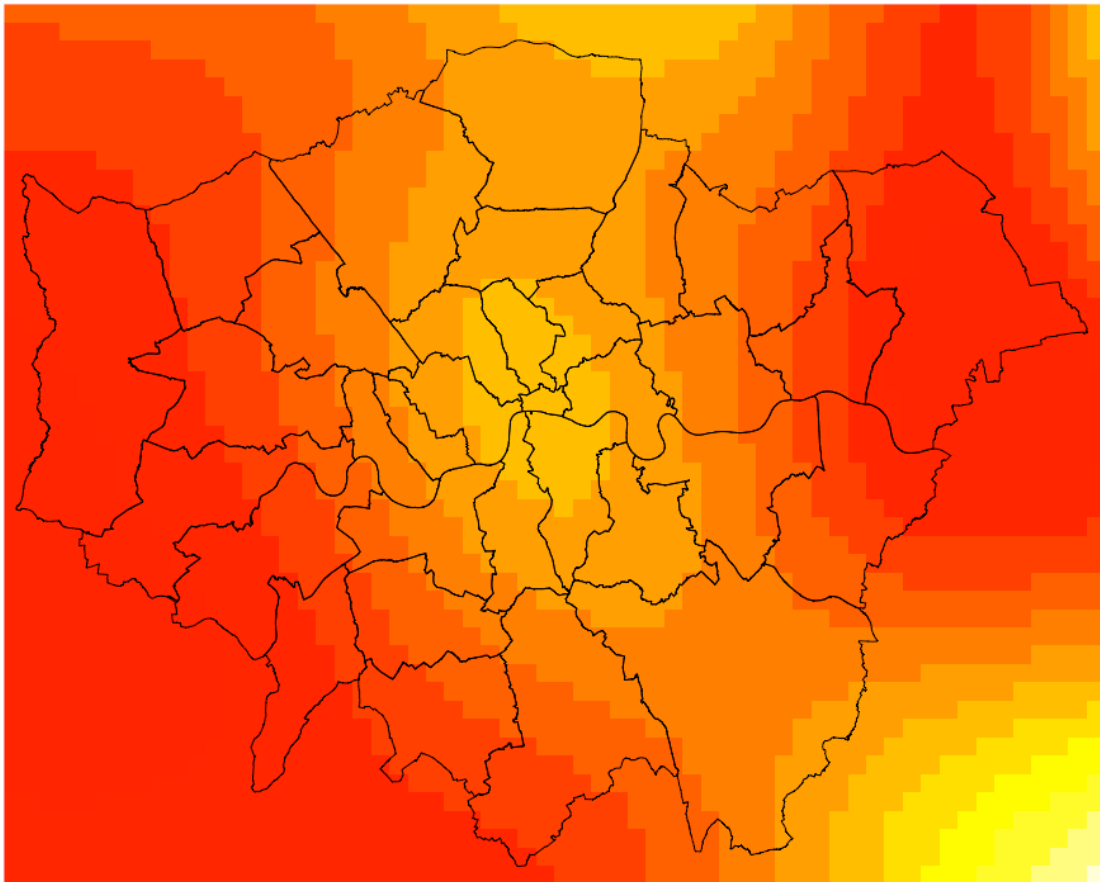
A contour plot is also possible:

```
plot(londonborough,border='lightgrey')  
contour(gwr.res$SDF, 'PROF',lwd=3,add=TRUE)  
plot(londonhp,add=TRUE,pch=16,col=adjustcolor('blueviolet',alpha.f=0.4))
```



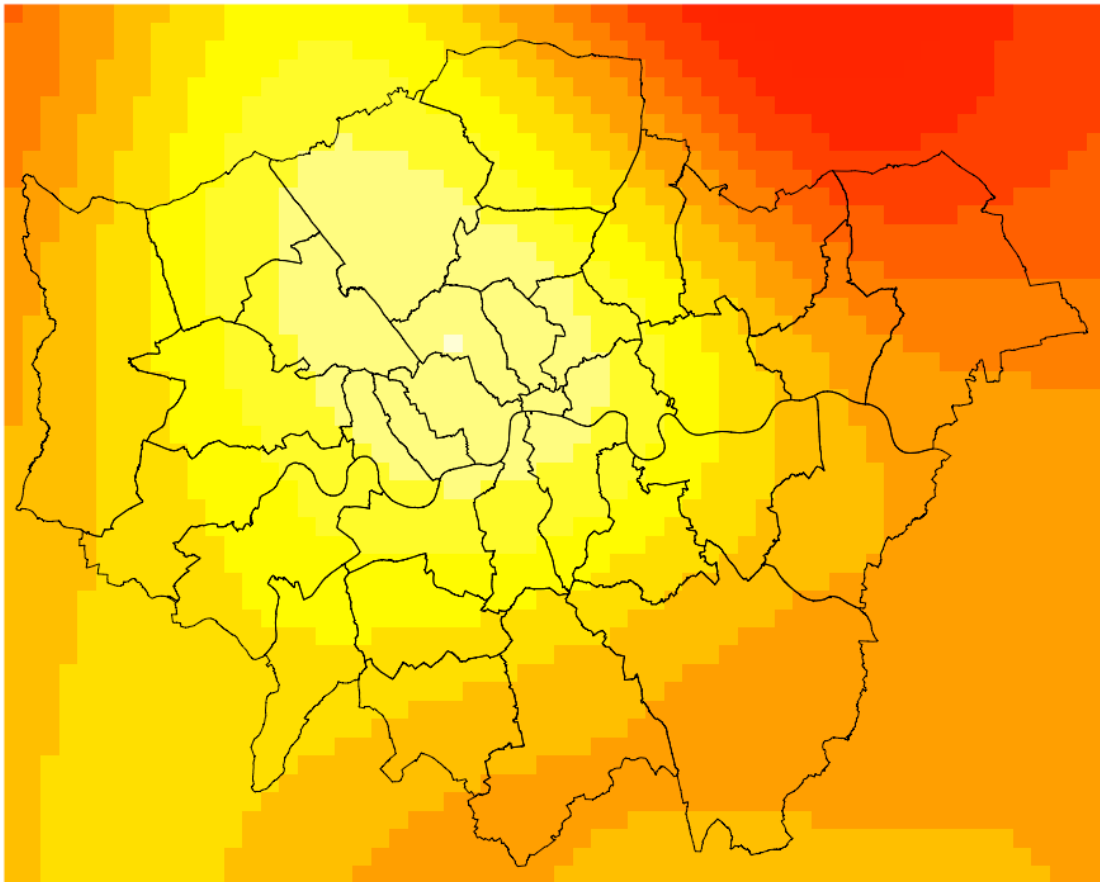
Both of these suggest the slope is greater in central north London. We may also wish to test how reliable these estimates are. One way to do this is to compute estimates of their standard errors via the `boot` package - essentially this creates bootstrap resamples of the local parameter estimates (Efron 1979), which may then have their standard deviations computed to obtain the standard errors.

```
library(boot)
set.seed(4676)
gwrcoef <- function(hpdf,i) gwr.basic(PPSQM~PROF, data=londonhp[i,], regression.
n.points=grd, bw=10000, dMat=DM[i,],kernel='gaussian')$SDF$PROF
bootres <- boot(londonhp,gwrcoef,100)
gwr.res$SDF$bsePROF <- sqrt(apply(bootres$t,2,var))
image(gwr.res$SDF, 'bsePROF')
plot(londonborough,add=TRUE)
```



It is also possible to estimate bias from bootstrap sampling by subtracting the mean of the bootstrap sampled estimates from the original estimate:

```
gwr.res$SDF$biasPROF <- bootres$t0 - apply(bootres$t,2,mean)
image(gwr.res$SDF,'biasPROF')
plot(londonborough,add=TRUE)
```



Further Ideas

1. You may want to produce contour plots of these bootstrap results to gain an idea of the magnitude of the estimated standard errors and bias.
2. You may want to produce larger bootstrap samples (say 1000 replications instead of 100).
3. Have a look at the bootstrap library help files - these will explain in more depth what the last few items of R code were doing.
4. To find out more about `GWmodel` see <http://www.jstatsoft.org/v63/i17> (<http://www.jstatsoft.org/v63/i17>)

References

Cleveland, William S. 1979. "Robust Locally Weighted Regression and Smoothing

Scatterplots.” *Journal of the American Statistical Association* 74 (368): 829–36. doi: 10.2307/2286407. ([https://doi.org/ 10.2307/2286407](https://doi.org/10.2307/2286407).)

———. 1993. “Visualizing Data.” New Jersey: Summit Press.

Efron, Bradley. 1979. “Bootstrap Methods: Another Look at the Jackknife.” *The Annals of Statistics* 7 (1): 1–26.